

# Блокировки

Виды блокировок

SQL И БАЗЫ ДАННЫХ

НЕДЕЛЯ 12

# План лекции

→ Транзакции и согласованность

→ Явные блокировки строк

→ Аномалии за рамками стандарта

→ Взаимоблокировки

→ Уровни изоляции  
и транзакционная модель

→ Специфические механизмы  
и архитектурные выводы

→ Диспетчер блокировок

# Транзакционные гарантии и подсистемы

A (Atomicity) — подсистема WAL (Write-Ahead Log).

C (Consistency) — фокус текущей лекции.

I (Isolation) — фокус текущей лекции.

D (Durability) — подсистема WAL (Write-Ahead Log).

## MVCC (многоверсионность)

Решает конфликты типа чтение-запись

Чтение никогда не блокирует запись,  
запись не блокирует чтение

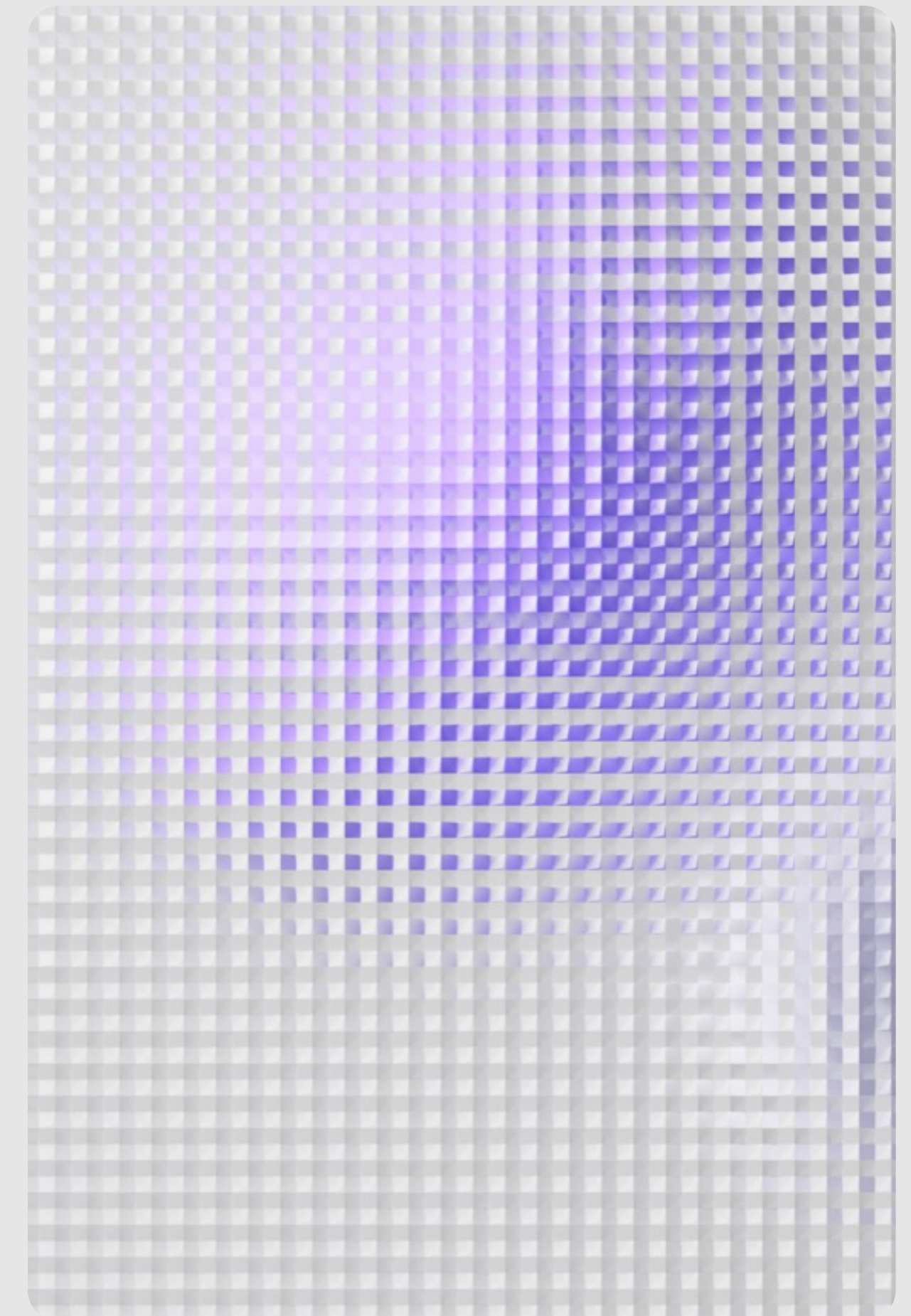
MVCC (многоверсионность)

## Lock Manager (диспетчер блокировок)

Решает конфликты типа запись-запись

Требует строгой сериализации  
и выстраивания в очередь ожидания

Lock Manager (диспетчер блокировок)



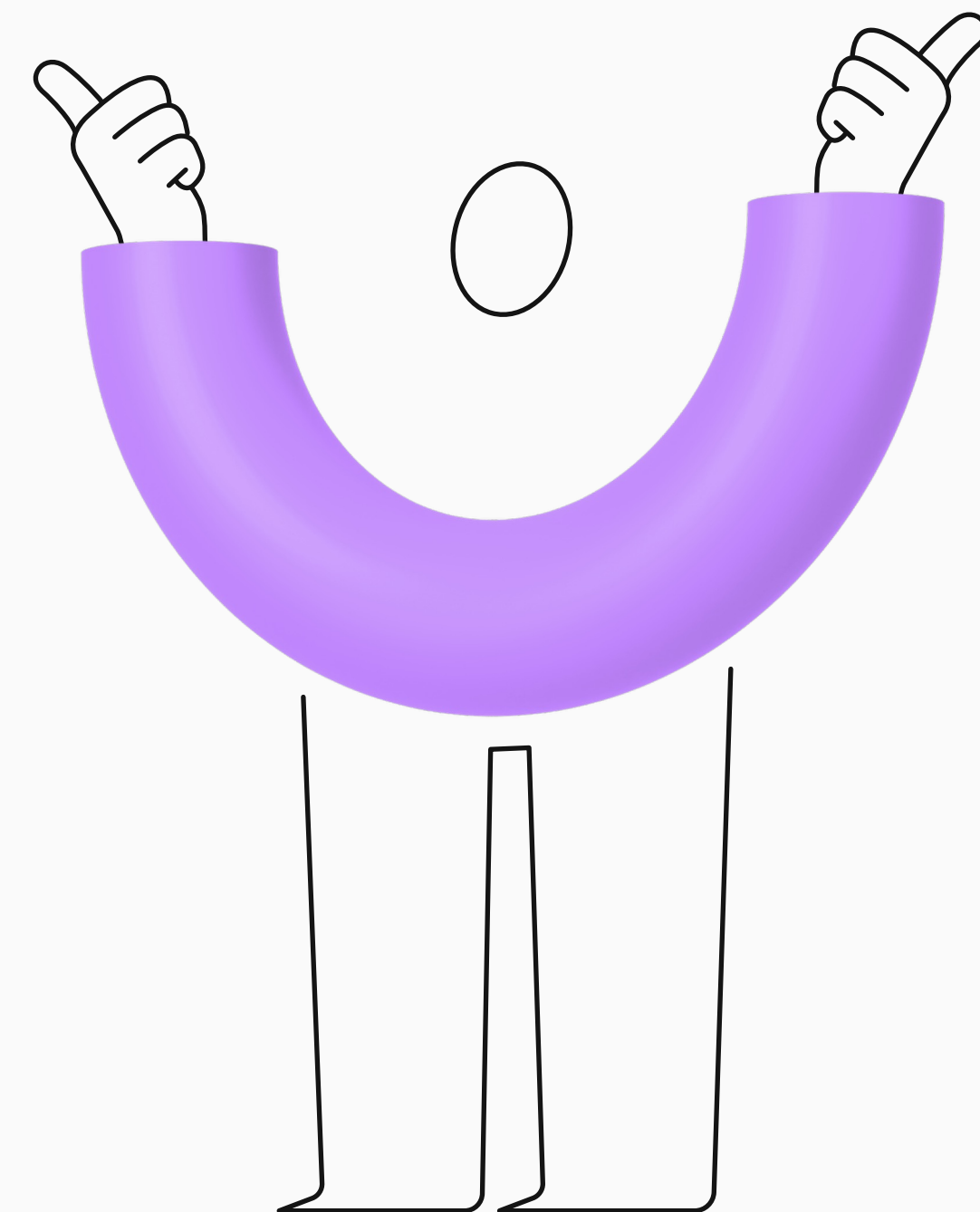


# Устройство Lock Manager в Shared Memory

→ PGPROC — структура процесса (backend), который отправил запрос.

→ LOCK — структура блокируемого ресурса (таблица, транзакция).

→ PROCLOCK — связующая структура. Показывает, какой процесс держит какой лок и в каком режиме.



# LWLocks (Lightweight Locks) и Spinlocks

## HEAVYWEIGHT LOCKS

|         |                               |
|---------|-------------------------------|
| Что     | Таблицы, строки, транзакции   |
| Цель    | Изоляция бизнес-логики (ACID) |
| Время   | Миллисекунды → часы           |
| Дедлоки | Обнаруживаются (timeout)      |

## LIGHTWEIGHT LOCKS / SPINLOCKS

|         |                              |
|---------|------------------------------|
| Что     | Буферы WAL, страницы         |
| Цель    | Защита структур С-кода в RAM |
| Время   | Наносекунды → микросекунды   |
| Дедлоки | Нет. Ведут к крашу инстанса  |

# Гранулярность и иерархия блокировок

## Cluster

---

Database

---

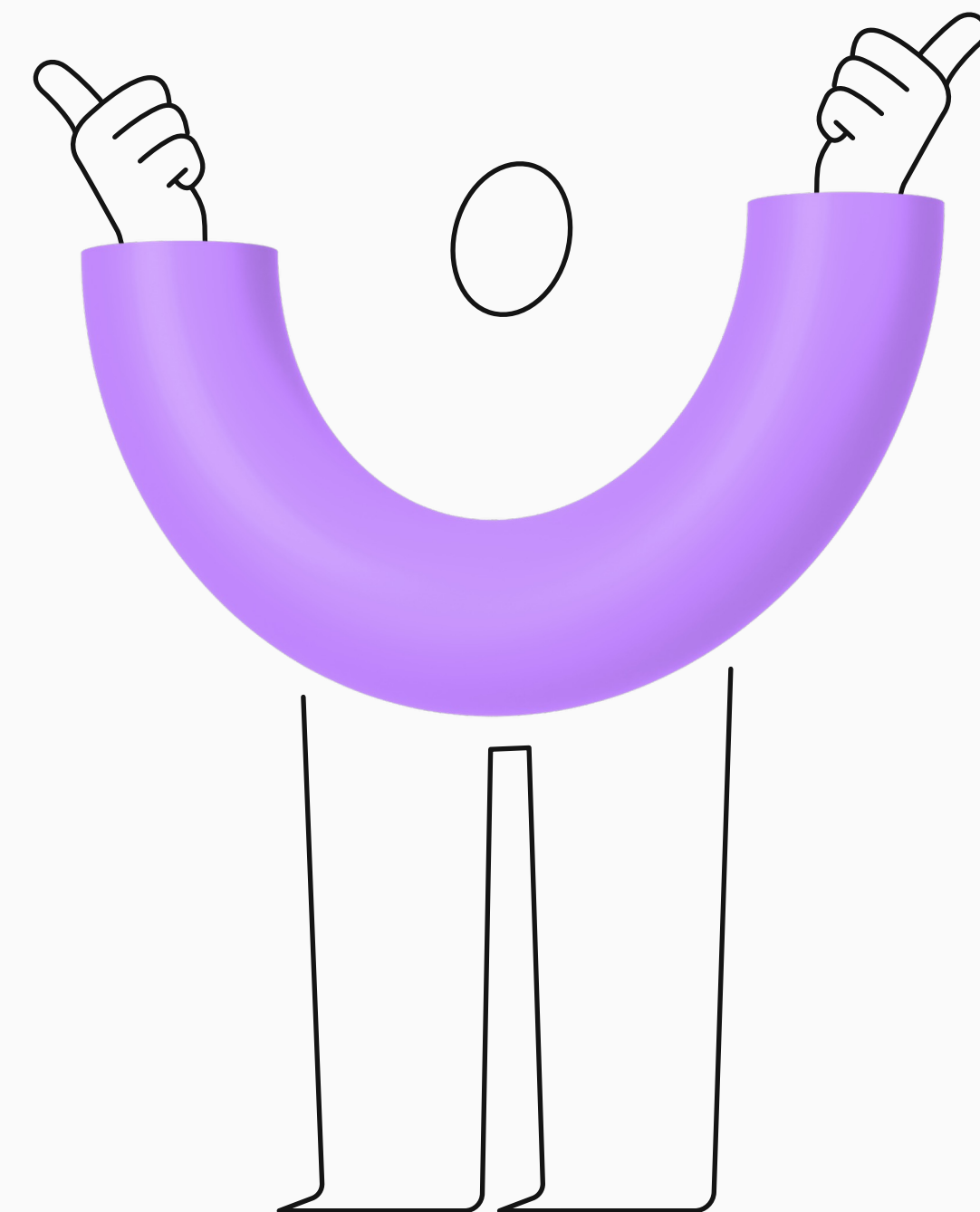
Relation (таблица)

---

Page (страница)

---

Tuple (строка)





# Матрица совместимости

| Запрошено/держит                 | Access Share<br>(SELECT) | Row Exclusive<br>(UPDATE) | Share<br>(CREATE INDEX) | Access Exclusive<br>(ALTER/DROP) |
|----------------------------------|--------------------------|---------------------------|-------------------------|----------------------------------|
| Access Share<br>(SELECT)         | ✓                        | ✓                         | ✓                       | ✗                                |
| Row Exclusive<br>(UPDATE)        | ✓                        | ✓                         | ✗                       | ✗                                |
| Share<br>(CREATE INDEX)          | ✓                        | ✗                         | ✓                       | ✗                                |
| Access Exclusive<br>(ALTER/DROP) | ✗                        | ✗                         | ✗                       | ✗                                |



# LWLocks (Lightweight Locks) и Spinlocks

Любой запрос наталкивается на несовместимость матрицы.

## Active

GRANTED QUEUE (удерживают замок)

---

|                             |                                                       |
|-----------------------------|-------------------------------------------------------|
| [PID: 1042]<br>ТРАНЗАКЦИЯ А | Режим: Access Exclusive<br>(выполняет<br>ALTER TABLE) |
|-----------------------------|-------------------------------------------------------|

## Sleep

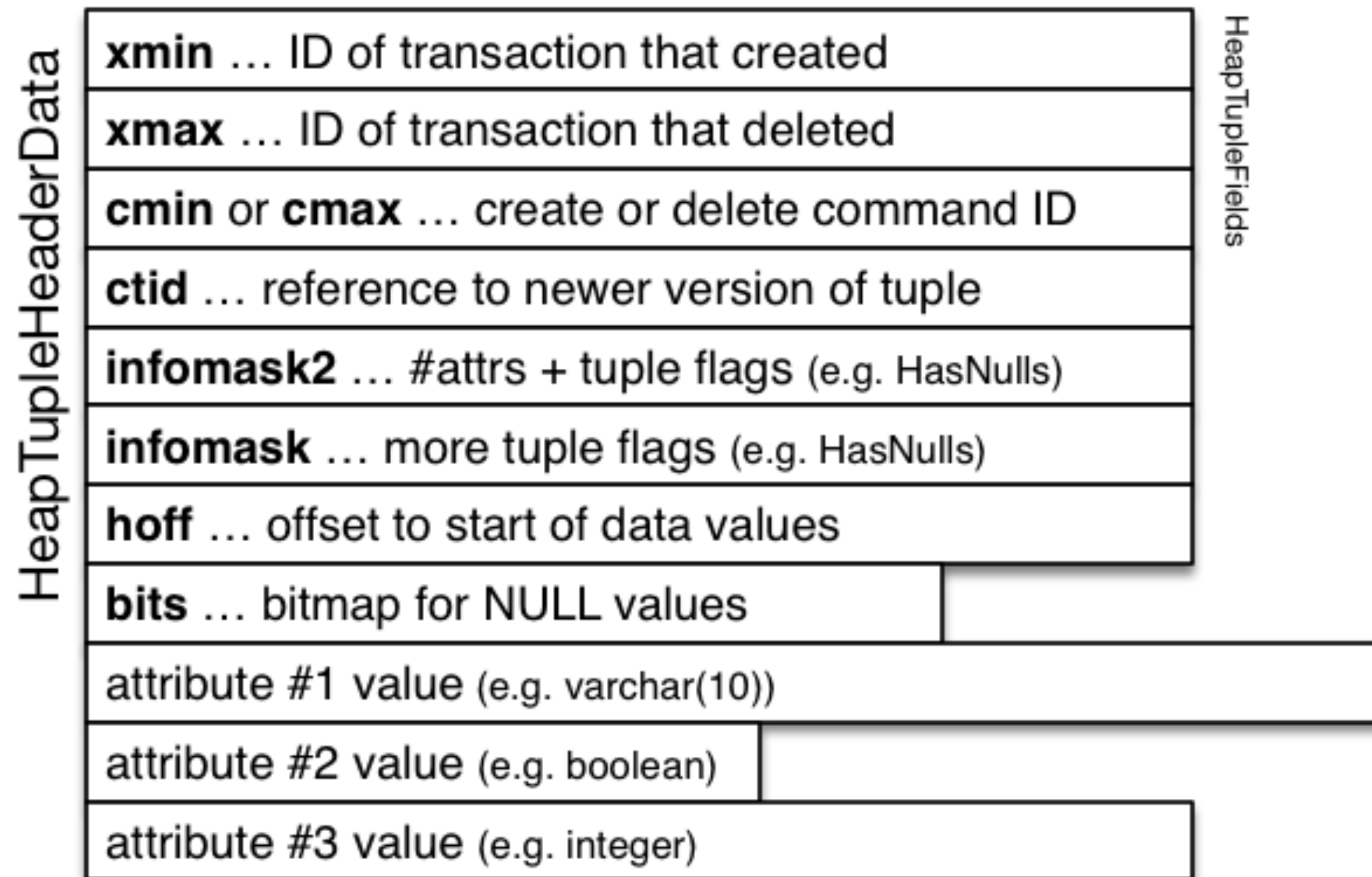
WAIT QUEUE (Очередь ожидания FIFO)

---

|                                |                                       |
|--------------------------------|---------------------------------------|
| 1. [PID: 2011]<br>ТРАНЗАКЦИЯ Б | Режим: Access Share (ждёт<br>SELECT)  |
| 2. [PID: 2045]<br>ТРАНЗАКЦИЯ В | Режим: Row Exclusive<br>(ждёт UPDATE) |
| 3. [PID: 2099]<br>ТРАНЗАКЦИЯ Г | Режим: Access Share (ждёт<br>SELECT)  |



# Архитектура Row-Level Locks



# Пессимистичная блокировка

```
SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;  
-- ROW BLOCKED
```

```
UPDATE accounts SET balance = balance + 100 WHERE id =  
1; COMMIT;
```

```
SELECT balance FROM accounts WHERE id = 1 FOR UPDATE;  
-- Wait Queue
```

```
-- Get the real value and install your lock
```

# For no key update

## Проблема

Таблица orders ссылается на таблицу users (FK).

- Если мы делаем `SELECT * FROM users WHERE id=1 FOR UPDATE`, СУБД жёстко блокирует строку пользователя.
- Если параллельно кто-то пытается вставить заказ для этого пользователя `INSERT INTO orders (user_id) VALUES (1)`, он зависнет, потому что проверка FK требует разделяемой блокировки ключа пользователя.

## Решение

- Введение `FOR NO KEY UPDATE`.
- «Я собираюсь изменить эту строку, но я клянусь, что не трону уникальные поля (ID)».



# Foreign Keys

## Сценарий каскадной блокировки:

- таблица orders имеет FK (user\_id) ссылающийся на users(id);
- T1 — делает INSERT INTO orders (user\_id=42, ...).

Чтобы гарантировать, что пользователь №42 не исчезнет, пока создаётся заказ, PostgreSQL неявно захватывает FOR KEY SHARE блокировку на строку.

## Пример конфликта:

- T2 пытается удалить пользователя 42 (DELETE FROM users WHERE id=42) и зависает;
- если она попытается изменить его первичный ключ (что редкость), она зависнет.

# SKIP LOCKED и NOWAIT

```
SELECT * FROM jobs WHERE status='new' LIMIT 1 FOR UPDATE
```

```
-- Это проблема
```

```
UPDATE jobs SET status = 'processing'
```

```
WHERE id = (
```

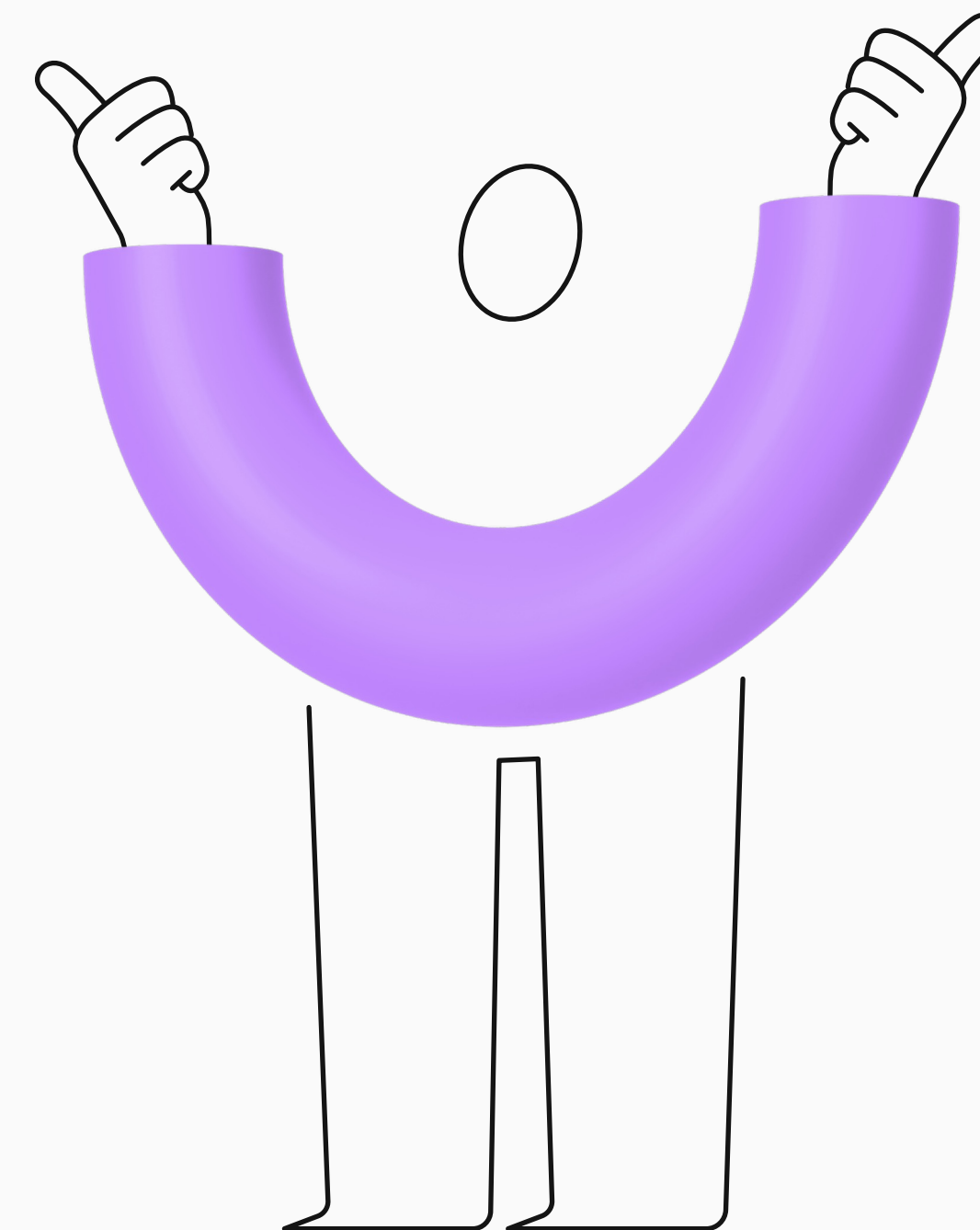
```
    SELECT id FROM jobs
```

```
    WHERE status = 'new'
```

```
    FOR UPDATE SKIP LOCKED
```

```
    LIMIT 1
```

```
) RETURNING *;
```



# Пессимистичная блокировка

| Запрошено/держит                 | Access Share<br>(SELECT) | Row Exclusive<br>(UPDATE) | Share<br>(CREATE INDEX) | Access Exclusive<br>(ALTER/DROP) |
|----------------------------------|--------------------------|---------------------------|-------------------------|----------------------------------|
| Access Share<br>(SELECT)         | ✓                        | ✓                         | ✓                       | ✗                                |
| Row Exclusive<br>(UPDATE)        | ✓                        | ✓                         | ✗                       | ✗                                |
| Share<br>(CREATE INDEX)          | ✓                        | ✗                         | ✓                       | ✗                                |
| Access Exclusive<br>(ALTER/DROP) | ✗                        | ✗                         | ✗                       | ✗                                |

```
ALTER TABLE users ALTER COLUMN id TYPE bigint;  
ALTER TABLE users ADD COLUMN rating INT DEFAULT calculate_rating();
```



# Опасные и безопасные миграции

→ **CREATE INDEX**

→ **CREATE INDEX CONCURRENTLY**

→ **DROP COLUMN**

→ **ALTER TABLE ... ADD COLUMN ...**



# SKIP LOCKED и NOWAIT

## GRANTED QUEUE (удерживают замок)

[PID: 101]  
Аналитика  
(тяжёлый  
SELECT)

Режим: Access  
Share

Время: 10m



```
BEGIN;  
  
SET LOCAL lock_timeout = '2s';  
  
ALTER TABLE orders ADD COLUMN status VARCHAR;  
  
COMMIT;
```

## GRANTED QUEUE (удерживают замок)

1. [PID: 205] Миграция (ALTER TABLE)                      Режим: Access Exclusive

Конфликтует с Access Share. Ждёт PID 101

2. [PID: 301] Клиент (Быстрый SELECT)                      Режим: Access Share

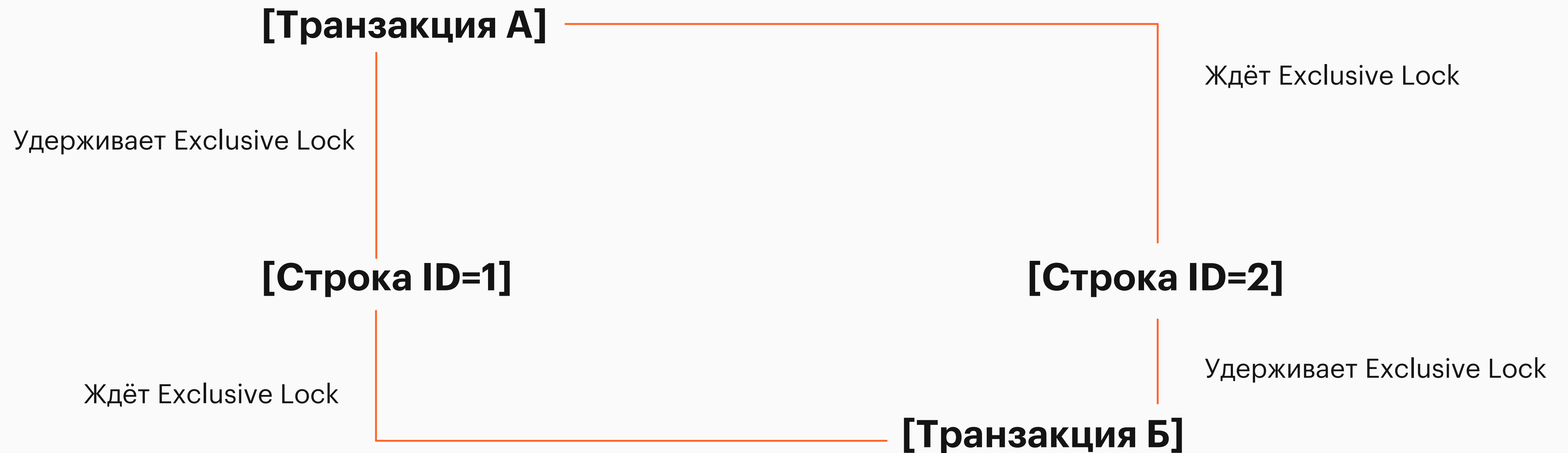
Блокируется миграцией по правилу FIFO

3. [PID: 302] Клиент (Быстрый SELECT)                      Режим: Access Share

Ещё 500 клиентских запросов  
(Пул соединений исчерпан, прод лежит)

# Взаимоблокировка

## Deadlock



# Алгоритм обнаружения и цена поиска

## Проблема:

- поиск циклов в направленном графе (DFS) — это дорогая операция для CPU. Если кластер обслуживает 10 000 RPS и при каждой блокировке СУБД будет строить граф и искать циклы, процессор сгорит.

## Решение:

- по умолчанию база считает, что любое ожидание — это просто медленный запрос;
- процесс покорно засыпает и ждёт;
- включается таймер `Deadlock_timer` (default = 1 second);
- если через 1 секунду замок не освободился, СУБД «просыпается», строит Wait-For Graph и ищет цикл.



# Опасные и безопасные миграции

→ Микросервис получает массив ID для обновления: [5, 1, 9]. Выполняет UPDATE в цикле: 5, затем 1, затем 9.

→ Перед любым массовым обновлением (или захватом FOR UPDATE) приложение обязано отсортировать ресурсы.

→ Параллельный запрос получает: [9, 5, 1]. Обновляет: 9, 5, 1.

→ Массив 1 сортируется: [1, 5, 9].  
Массив 2 сортируется: [1, 5, 9].

→ Результат:  
пересечение порядков = Deadlock.

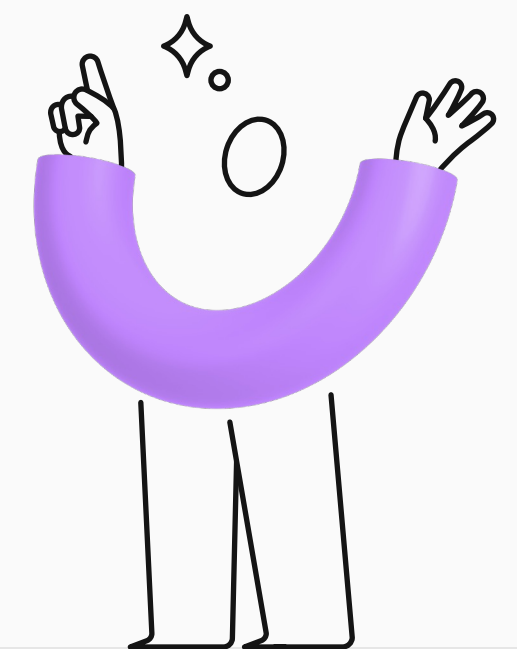
→ Результат: транзакция Б попытается взять ID=1, увидит блокировку от транзакции А, и подождёт. Цикл невозможен математически.

# Foreign Keys

Сценарий: запуск ежедневного отчёта (Cron-задача) на кластере из 10 серверов. Вы берёте `pg_try_advisory_lock(12345)`. Кто успел, тот выполняет расчёт.

Проблема: разработчики часто используют сессионные блокировки (`pg_advisory_lock`), думая, что при закрытии транзакции замок исчезнет.

PgBouncer (Transaction Mode): в микросервисах коннекты переиспользуются пулером. Если вы взяли сессионный лок, сделали COMMIT и вернули коннект в пул, замок остается висеть. Следующий микросервис, получивший этот коннект, не сможет работать или неожиданно заблокирует других.



# Опасные и безопасные миграции

## pg\_stat\_activity

---

pid (ID процесса)

---

state (active/idle)

---

wait\_event\_type (Lock)

---

wait\_event (зависел от...)

---

query (текст SQL-запроса)

JOIN

## pg\_locks

---

pid (ID процесса)

---

locktype (relation/transaction)

---

mode (AccessExclusiveLock)

---

granted (true/false)

---

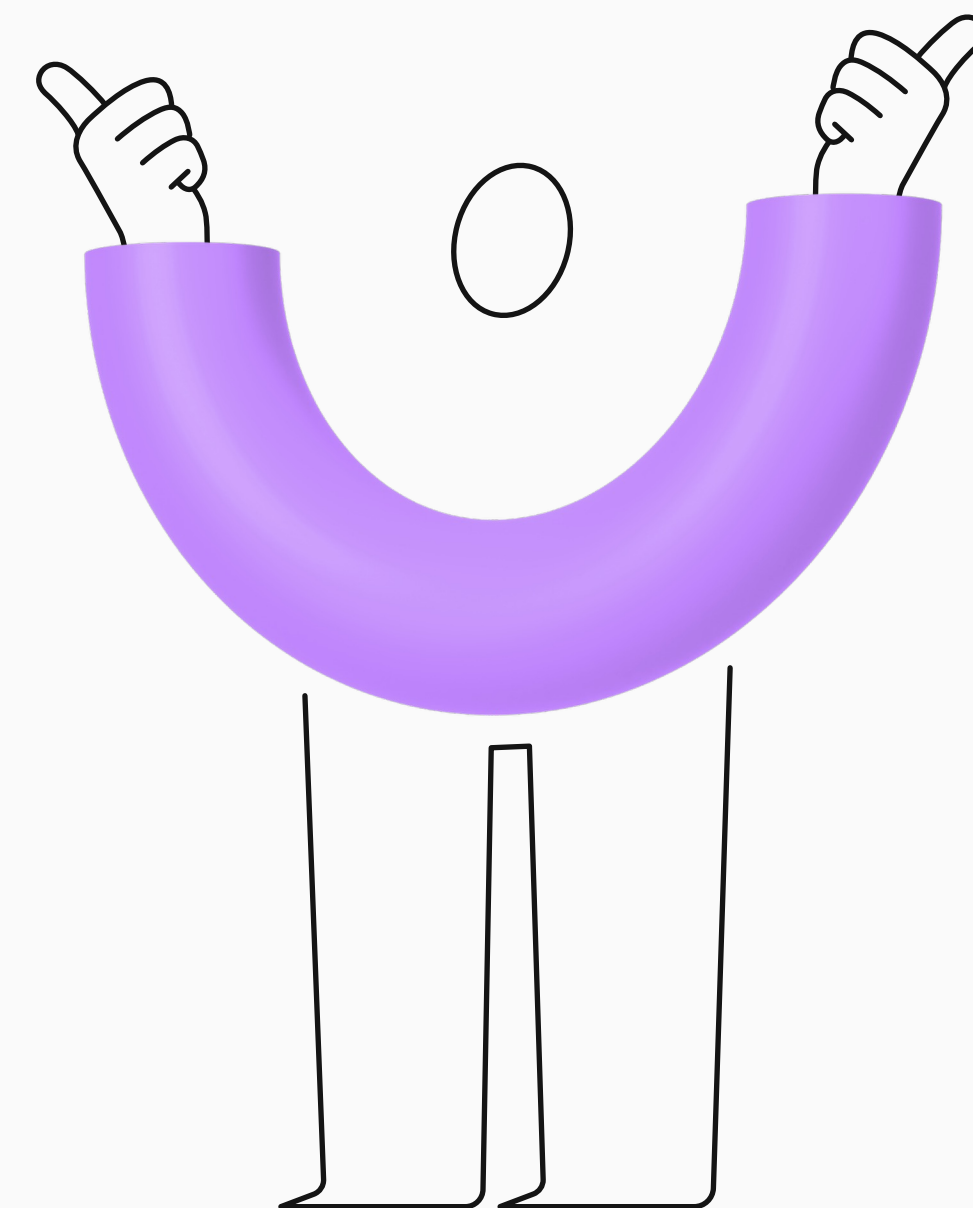
relation (ID таблицы)



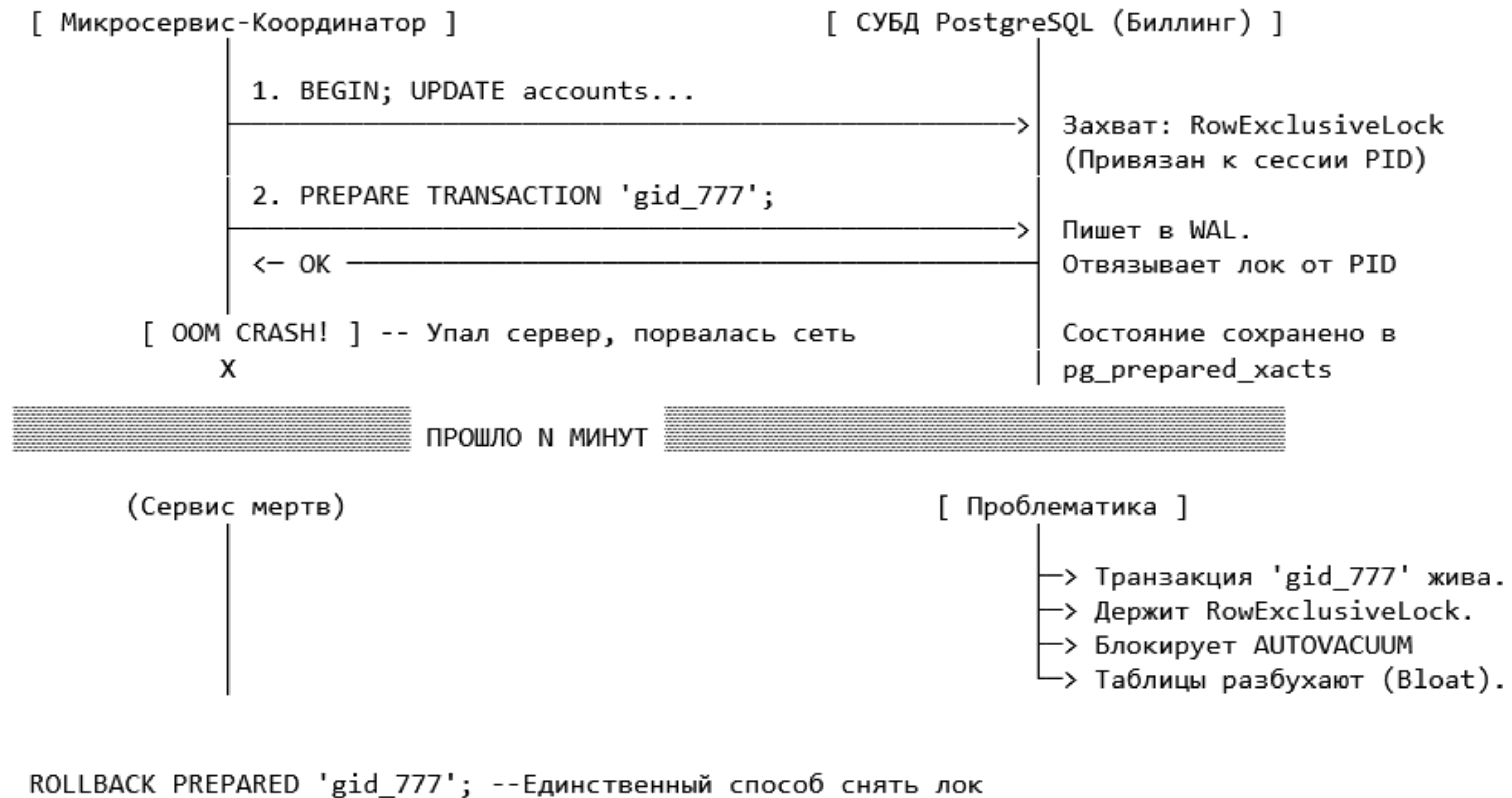
# Траблшутинг

## Устранение корневого блокиратора

```
SELECT
    pid,
    state,
    pg_blocking_pids(pid) AS blockers,
    query
FROM pg_stat_activity
WHERE wait_event_type = 'Lock';
```



# Two-Phase Commit





# Резюме

→ Keep it short

→ Сортировка ресурсов

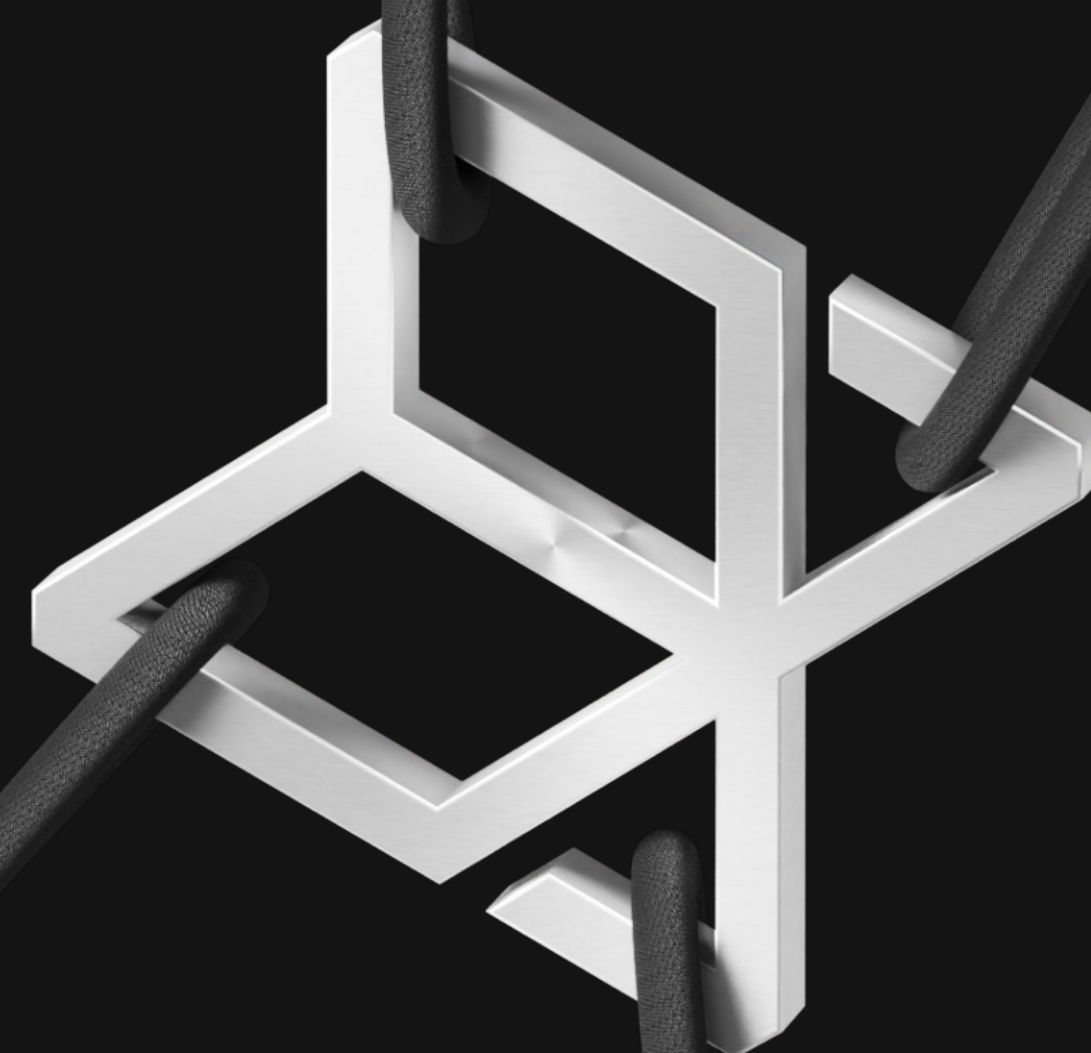
→ Защита DDL

→ Изоляция снимков

→ Очереди



Вопросы



## Минутка обратной связи

